

Hardware-Conscious Performance Engineering of Fainder for Distribution-Aware Dataset Search

Master's Thesis Defense

Tarik Abu Mukh

August 23, 2026

Technische Universität Berlin

Chair of Database Systems and Information Management (DIMA)

Reviewers: Prof. Dr. Volker Markl, Prof. Dr. Matthias Böhm Advisor: Lennart Behme

Problem

Method

Results: Four Ceilings

Results: Negative Composability

Results: Comparison with Python

Related Work

Conclusion

Problem

- Raw data not searchable (privacy, licensing, scale) → per-column **histograms**
- Predicate class: “ p -th percentile $\bowtie$ threshold τ ”
- **Fainder** [Behme et al., VLDB 2024]:
 - K-means clusters → shared bin grid per cluster → precomputed cumulative densities
 - Percentile predicate $\Rightarrow$ **binary search**
- GitTables, 5M histograms: >2 orders of magnitude vs profile scan, 1 TB $\rightarrow$ 3.2 GB

This thesis

How does Fainder’s query phase run on a modern multi-core server, and what makes it fast?

Fainder in Plain Words



- Histogram = “how many values fall in each range”
- One family, one x-axis: the same question lands at the same position everywhere
- Stored per x-axis point: the fraction of each column’s values below it — **sorted**, with IDs
- Sorted = binary-searchable: ~191 short lists, never 5M summaries
- “median > 20” $\iff$ “fraction of values ≤ 20 is < 0.5”: τ picks the list, p picks the cut
- Clusters are independent $\rightarrow$ parallel. Search = few dependent reads; emit = many writes

The Python Baseline

- GIL: one thread at a time; searchsorted never releases it
- Thread sweep **flat**: 2% spread at any t
- AoS layout: unread ID next to every value
- Interpreter dispatch on every iteration

10,000 queries, 5M histograms
~2.4 hours

(8,589 s, any thread count)

- IPC 2.35 vs Rust's 0.96 — yet $378\times$ slower: $6.2\times$ the instructions, one effective core
- Rust wins by executing fewer instructions across all cores in parallel, not by winning on IPC

Goal and Scope

- Close the algorithm–implementation gap; query results unchanged
- Rust query engine behind the existing Python API (PyO3); >20 optimisation axes ablated
- Constraint: bitwise-identical results, precision = recall = 1.000, every build
- Out of scope: algorithm changes, GPU, distributed execution

The investigation is the thesis, the port is its instrument

Four hardware ceilings + negative composability + a pre-flight methodology.

Method

Experimental Setup

- 2× Xeon Platinum 8468H (Sapphire Rapids): 96 cores, 105 MiB L3/socket, 1 TB DDR5
- 48 cores per socket $\rightarrow t \leq 48$ stays on one socket

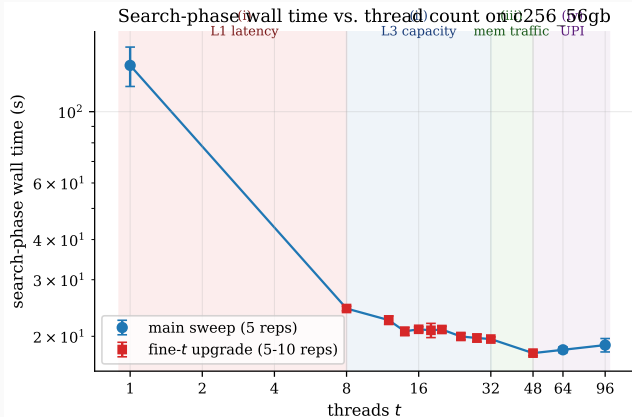
Dataset	n_{hist}	K_{eff}	hist/cluster	role
c256_10gb	324 K	129	~ 2.5 K	small
c256_30gb	997 K	184	~ 5.4 K	medium
c256_56gb	5.02 M	191	~ 26 K	primary
c1024_56gb	5.02 M	610	~ 8.2 K	OOD ($3.2\times$ sparser)

- Same 10K queries everywhere; 5-rep medians; $t \in \{1 \dots 96\}$; perf counters per cell
- 2,658 runs; every claim traces to a benchmark-database row

- PyO3 extension module; clustering and loading stay in Python
- Rayon work-stealing over clusters (idle threads take work); no GIL in the hot path
- One Cargo feature flag per axis → every variant is a separate measurable build:
 - layout: SoA / AoS, f16, packed IDs search: AVX-512, 8-way batch, lockstep, PGM
 - scheduling: column-centric, query-batch, morsel, pinning memory: pooled, mimalloc, NUMA
- Every build validates bitwise-identical against the reference

Results: Four Ceilings

Finding 1: Four Performance Ceilings



1. Single reads ($t \leq 8$): **refuted**
2. Shared L3 ($t \approx 16$)
3. Memory-traffic pressure ($t \in [32, 48]$)
4. Cross-socket link ($t > 48$)

One regime each.

One intervention class each.

Ceiling (i): Single-Thread Latency, Refuted

What it is: binary search is a **dependent-load chain** — probe $i+1$ cannot start until probe i returns. The literature's default bottleneck for in-memory search.

Ablation	Idea	vs default, all 18 cells	
simd	vectorise the last ~ 16 steps	0.90–1.07×	noise
batch-search	overlap 8 searches' loads	0.96–1.09×	noise
horizontal-simd	16 queries walk in lockstep	0.95–1.12×	noise
pgm	predict the position, skip the chain	0.91–1.13×	noise

- Counters at $t=1$: IPC ≈ 1.9 , L1d miss/load 0.5–0.7% — the chains are cache-resident
- PGM verifiably cuts chain depth to $O(\log \log n)$; the wall does not move
- The wall is the *number* of chains (one per cluster), not the depth of any single chain

Ceiling (ii): Shared L3 Capacity

The claim: at $t \in [8, 32]$ the threads' combined working sets overflow the shared 105 MiB L3.
Two probes (< 1 = faster than default, primary workload):

Build	$t=1$	$t=8$	$t=16$	$t=32$	$t=64$	$t=96$
aos (capacity probe: doubles bytes/line)	1.03	1.17	1.02	1.00	0.96	0.99
pooled (allocator probe: no per-cluster allocs)	0.53	0.66	0.80	0.95	0.93	1.02

- aos changes *only* footprint; its penalty sits inside the window (+17% at $t=8$, $\sim 2\times$ L3 working set) and vanishes outside $\rightarrow$ capacity is real in $[8, 32]$
- Yet the biggest win in that window fixes the allocator's lock, not capacity — and it fades ($0.66 \rightarrow 0.95$) as ceiling (iii) takes over
- f16 (halves bytes/value): null on the dense primary, pays $\sim 20\%$ at the sparse OOD density — footprint only matters where it actually binds

Verdict: one regime, two mechanisms — allocator queueing on dense clusters, L3 footprint on sparse.

Ceiling (iii): On-Socket Memory-Traffic Pressure

What it is: all cores share the queues on the path to memory (L3 $\rightarrow$ mesh $\rightarrow$ memory controller). Each added core lengthens every queue, so every access waits longer — while total traffic sits at $\sim 0.5\%$ of STREAM peak. **Congestion, not saturation.**

The signature, c256_30gb, wall in seconds:

Build	$t=64$	$t=96$	step	
default	4.15	5.89	+42%	(IPC 1.93 $\rightarrow$ 0.55, LLC misses $\times 1.6$)
packed-ids	3.88	4.70	+21%	(same step, half the damage)

- Anti-scaling: adding 32 cores makes the run 42% *slower*
- Fix class = fewer bytes per operation: packed-ids emits 20-bit instead of 32-bit IDs (-37% emit bytes) $\rightarrow$ -20% at the anti-scaling cell

Ceiling (iv): Cross-Socket UPI Interconnect

Same workload, four memory placements (numactl probe, wall in seconds):

Placement	meaning	$t=32$	$t=48$	$t=64$	$t=96$
A: unpinned	OS default, first-touch	19.28	16.97	18.19	17.58
B: single socket	cores + memory on socket 0	16.77	15.04	–	–
C: interleave	pages alternate sockets per 4 KB	25.87	23.73	22.91	24.83
D: cross-socket	memory socket 0, cores socket 1	19.08	16.86	–	–

- B vs A: removing all cross-socket traffic saves **11–13%** — that saving *is* the link cost
- C vs A: “fair” page balancing costs **26–41%**: half of all accesses land remote, and the per-page alternation defeats the prefetcher
- $D \approx A$: even the worst case matches default $\rightarrow$ the baseline was already paying the link

Results: Negative Composability

Finding 2: Optimisations Compose Negatively

Five pairs, five axes, one shape: *the coarser optimisation absorbed the headroom; the finer one's overhead becomes net cost.*

Composition	Intended axis	Outcome
best-combo + packed-ids	emit-ID bandwidth	NEG 12/18 cells, up to +24%
best-combo + qbatch $K=128$	cold-load amortisation	NEG every cell, +8 to +32%
qbatch + morsel	cross-worker coordination	NEG every cell, +8 to +12%
packed-ids + local-ids	read-side bandwidth	NEG +186 to +245%
first-touch + interleave	memory-locality balance	NEG every cell, +26 to +41%

- Every pair had a plausible positive argument; every one falsified by perf counters
- Five axes, one shape: structural, not coincidence

Finding 3: Pre-Flight Ceiling Identification

Classical ablation (build $\rightarrow$ measure $\rightarrow$ keep) assumes optimisations are independent. Here they are not: what helps depends on the binding ceiling *at the target composition*.

Before writing code:

1. Probe the binding ceiling at the target composition (perf)
2. Does the ablation's axis still bind there? If not: stop
3. Predict the saving (bandwidth-budget arithmetic)
4. Measure where the ceiling binds; separate saving from overhead
5. A negative composition counts as a result

Applies to any hardware-near system with stacked ceilings.

Dispatch on Regime

- No dominating build $\rightarrow$ per-regime selection: $(n_{\text{hists}}, t) \rightarrow \text{build} + \text{flags}$
- best-combo at moderate t / big data; packed-ids at $t=64$; qbatch at $t=96$; columnar at low t ; default on small data

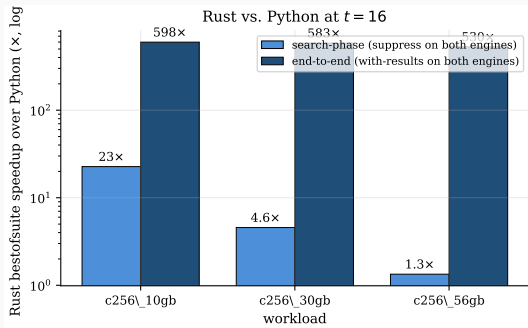
Validation:

- In-distribution: 17/18 exact, 18/18 within 5%
- Out-of-distribution ($3.2\times$ sparser, never tuned on): 6/6 within 1%
- Combined: 23/24

Boundaries are machine-specific; the probing procedure is not.

Results: Comparison with Python

Speedup over the Python Baseline



Workload	search	end-to-end
c256_10gb	22.7×	598×
c256_30gb	4.6×	583×
c256_56gb	1.34×	530×

- End-to-end: 2.4 h $\rightarrow$ 16 s
- $\sim 99\%$ of the ratio = avoiding Python's result-dict construction
- Search alone bends toward the shared ceilings
- Rebinning kernel: $4.7\times$ / $2.2\times$ on construction

Related Work

Positioning Against the State of the Art

- Layouts: Manegold et al. [2000] — memory access dominates → our SoA; ART, Eytzinger, learned indexes target cache behaviour
- On heterogeneous, cache-resident arrays: classical binary search wins (consistent with ceiling (i))
- Execution: Neumann [2011] compile-don't-interpret → the port; Leis et al. [2014] → work-stealing
- Composability traditions (Selinger [1979]; Caladan [2020]; memory-wall) assume gains compose
- No prior documentation of five mechanism-grounded negative pairs + a falsifiable pre-flight check
- Gap filled: first hardware characterisation of histogram-based percentile search; first horizontal-SIMD measurement on Sapphire Rapids

Conclusion

1. Rust re-implementation: bitwise-identical, $530\text{--}598\times$ end-to-end, PyO3-integrated
engineering
2. Four-ceiling characterisation on Sapphire Rapids, via perf *scientific*
3. Negative-composability pattern: five instances, five axes *scientific*
4. Pre-flight ceiling identification *methodological*
5. Dispatch-on-regime policy, 23/24 within 5% incl. OOD *engineering*

(1) and (5) are the staging ground; (2)–(4) generalise.

- (F1) Transparent Huge Pages: $512\times$ fewer TLB misses per covered page $\rightarrow$ targets the local-ids mechanism
- (F2) NUMA-aware build for $t > 48$: shard clusters across sockets, node-local workers
- (F3) Adaptive precision: f32 at low t , f16 at high t / sparse density, selected at runtime
- Column-centric: widen its $t=1$ win ($2.17\text{--}2.63\times$) by fixing cross-cluster imbalance
- Dispatch validation beyond the $\sim 8.2\text{K--}26.3\text{K}$ hist_s/cluster range

Closing Remark

- The speedups are specific to this codebase, machine, and workload
- What generalises: whether optimisation B helps depends on which ceiling A already collected — invisible to per-operator cost models

Closing remark

Identify the binding ceiling at the target composition before committing the implementation cost of the next optimisation.

Questions?

Backup

Backup: One Query, End to End

Query: “ $median > 20$ ” = ($p=0.5$, $>$, $\tau=20$)

$$median > 20 \iff CDF(20) < 0.5$$

Cluster A index (toy: 3 histograms)

bin edges: [0, 10, 20, 30, 50]

edge	sorted CDFs	IDs
≤ 10	[0.00, 0.10, 0.20]	H2 H1 H5
≤ 20	[0.25, 0.45, 0.50]	H2 H5 H1
≤ 30	[0.75, 0.75, 0.80]	H2 H5 H1

1. Binary search $\tau=20$ in the bin edges $\rightarrow$ column “ ≤ 20 ”
2. Binary search 0.5 in that sorted column $\rightarrow$ cut after 2 entries
3. Emit the IDs before the cut: **H2, H5**
(H1 sits exactly at 0.50: median = 20, not > 20)
4. Cluster B (salaries, range $\geq 10\,000$): τ below range $\rightarrow$ **all match, no search**
5. Union across clusters = the answer

Clusters are independent $\rightarrow$ the cluster loop parallelises. Search = few dependent reads; emit = many writes.

Backup: Why is end-to-end $530\times$ if search is only $1.34\times$?

- Python's per-(query, cluster) result-dict construction dominates its wall. Measured at $t=16$, materialisation accounts for:
 - c256_10gb: 580.9 s of 599.7 s (96.9%)
 - c256_30gb: 1,878.5 s of 1,890.3 s (99.4%)
 - c256_56gb: 8,568 s of 8,589 s (99.8%)
- Rust emits results without per-cluster dict construction, so it avoids this cost entirely.
- We therefore report both columns: suppress-vs-suppress isolates the search engines, with-results-vs-with-results is what a user experiences.
- Conflating the two would attribute Python's materialisation cost to Rust's search engineering.

Backup: Why does f16 win nothing standalone?

- Prediction: halve the footprint, win in the regime where the f32 working set exceeds L3 but the f16 one fits.
- On c256_56gb the per-cluster f32 slice (~ 105 KiB) is already L2-resident. Halving it unlocks no new cache-resident regime, while every comparison pays the f16 $\rightarrow$ f32 dequantisation.
- Result: no significant standalone win in the 10-rep fine- t sweep, and a +15.6% drag inside the bundle at $t=16$.
- On c1024_56gb ($3.2\times$ sparser, ~ 33 KiB slices, 610 clusters) the aggregate cluster-loop footprint binds. There f16 carries $\sim 20\%$ of the bundle win.
- Footprint optimisations are density-conditional.

Backup: Why “memory-traffic pressure” and not “DRAM bandwidth”?

- STREAM Triad on this machine reaches ~ 330 GB/s aggregate.
- Accounting the workload’s LLC-miss traffic at 64 B per missed probe puts it near 0.5% of that peak.
- “Bandwidth-bound” claims saturation of the DRAM channels. The counters show nothing close to saturation.
- What we observe is anti-scaling from aggregate pressure on the on-socket memory subsystem: queue occupancy and allocator traffic, at utilisation far below peak.
- The name matters because it predicts which interventions help: fewer bytes per comparison and per emit. A pure bandwidth upgrade would not.

Backup: The local-ids regression

- local-ids (per-cluster reindexing) was pre-registered as a positive prediction: narrower IDs, less read-side bandwidth.
- Measured: **+186 to +245%** at $t \in [8, 64]$ on top of packed-ids. The prediction is falsified.
- Mechanism, from a direct perf stat probe: the $\text{bw} \approx 13$ decoder's access pattern over the packed buffer raises the dTLB miss rate $52\times$ per instruction.
- TLB-walk stalls at ~ 100 cycles each drown the bandwidth saving.
- Future work (F1): Transparent Huge Pages (2 MiB pages) cut TLB-miss frequency $512\times$ per covered page. That probe is the natural follow-up.

Backup: What generalises beyond this machine?

- The thresholds do not generalise. The methodology does.
- All experiments ran on one Sapphire Rapids 8468H configuration. Genoa, Granite Rapids, or Grace would shift per-ceiling thresholds and dispatch boundaries.
- The pre-flight procedure (probe the binding ceiling at the target composition) is machine-agnostic. The four-ceiling taxonomy is a hypothesis set to re-test on new hardware, and not a constant.
- Density scope: dispatch is validated between $\sim 8,200$ and $\sim 26,300$ hists/cluster. Outside that range the policy is a prediction.
- A multi-machine deployment would add a fifth candidate ceiling (network) with its own intervention class.

Backup: How was correctness validated?

- Every Rust build validates bitwise-identical against the Python reference across all workloads: precision = recall = 1.000.
- Validation runs inside the benchmark harness, so each measured configuration is also a correctness check. Suppress mode is a stopwatch setting only; correctness is always checked with results on.
- Rebinning kernel: verified against the Python reference on 200 queries. Rust rounds half-away, NumPy uses banker's rounding. The maximum cumulative difference is 10^{-4} at a handful of rows, with no effect on sort order or query results.
- The kernel falls back to the Python pipeline if the extension is not importable or for `cubic_spline` bin estimation.

Backup: Why Rust, and not C++, Cython, or Numba?

- The requirements: no GIL in the hot path, explicit control over layout and allocation, safe shared-memory parallelism, and a maintainable Python integration.
- Rust with PyO3 covers all four: Rayon work-stealing without data races, Cargo feature flags that make every ablation a separate reproducible build, and a drop-in extension module behind the existing API.
- C++ could reach the same performance. The feature-flag ablation matrix and memory safety under 96-thread parallelism are where Rust paid off in practice.
- Numba and Cython remove less of the interpreter overhead and leave allocator and layout decisions with Python. Our measurements show those layers are the actual cost.

Backup: Why does the rebinning speedup drop at 5M histograms?

- $4.72\times$ (324 K) and $4.77\times$ (997 K) drop to $2.18\times$ at 5.02 M histograms.
- Cause: the kernel's own parallelism ceiling. At that scale it runs at ~ 2 effective cores of 96, with $\sim 46\%$ of kernel time on a shared per-cluster allocation.
- An earlier hypothesis (NumPy bulk operations dominating on the Python side) was tested and refuted.
- This is a construction-time cost, paid once per index build. The query-phase findings are unaffected.